Dynamic Typing Project

CS-3180

Perils

- Perils of dynamic typing
- Causes a lot of problems
- Seemingly Ill-defined behavior

Benefits

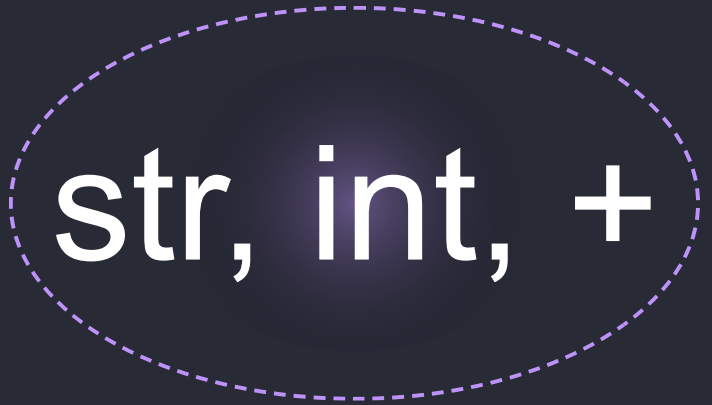
- Dynamic typing is not *entirely* evil
- As much as it pains me to say
- Let's build a project:
- Practice more python
- Take advantage of dyn. typing

Calculator

- Ever used DESMOS?
- Desmos supports variables
- Aim for something similar
- Minus any of the graphing stuff

Supports:

- Strings
- Numbers
- Variables
- Addition
- Multiplication? (time permitting)



Input/Output

- Input: data from a file
- Parse it line-by-line
- Output: State of all variables

Design

- Before we can start this
- Spend time designing our input
- Call this input our "specification"
- Hopefully, design should be easy to parse
- First, define our syntax

Design

- Easiest to just show you the goal
- Then, we can work backwards
- Show some simplification

```
a:5
b:6
c:a + b
d:'hello'
e:d + a
```

Output

```
a:5
b:6
c:a + b
d:'hello'
e:d + a
```

```
{
    'a': 5,
    'b': 6,
    'c': 11,
    'd': 'hello',
    'e': 'hello5'
}
```

Start Small

- Read the file
- Would like to get this in a more useful format
- Get rid of newlines
- Useful features:
- with, open(), as, readlines(), strip()

Comprehensions

- Once we've read the lines
- May need to preprocess them

```
for line in lines:
    line = line.strip()
```

- But *line* is not a reference to its original data
- It's a copy

Comprehensions

- Iteration Abstracts are common in high level languages
- Python's are called Comprehensions
- result = [(OUTPUT) (ITERABLE) (CONDITIONAL)]
- Similar things exist for dictionaries

```
[x+2 for x in range(0,10) if x%2 == 0]
```

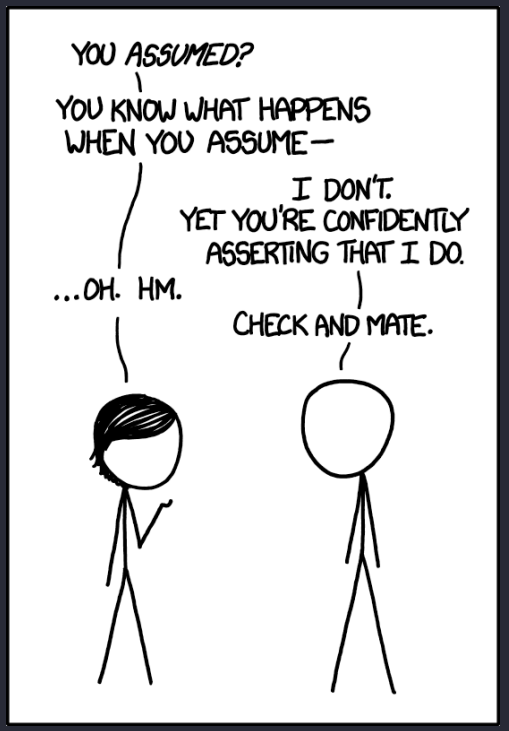
Give me this | in this collection | under these conditions

Vocabulary

- Literals:
- Fixed value in source "code"
- Expressions:
- Bits of code that will evaluate to a result
- Variables:
- Symbolic label for data 🤓
- https://xkcd.com/1339/

Simplifying Assumptions

- Expressions -> Always two variables
- New Line -> New variable
- Only supporting + operator (?)
- Only Support Strings(' ') and int
- Assume correct usage
- https://xkcd.com/1339/



Line Processing

- Delimiter:
- A boundary character
- Our ":" delimits:
- What part is a variable
- And what part is data

```
a:5
b:6
c:a + b
[VARIABLE]: [LITERAL/EXPRESSION]
```

Two Cases

- After a variable
- The data is EITHER:
- Expression -> Evaluate -> Store
- Literal -> Store

- Let's also create a place to store things

Python Dictionaries

- Declared with {}
- Store Key-Value pairs
- "Associative Array"
- We'll likely need an empty one
- Use global keyword for outside scope

```
fav_colors = {
    "Alice": 1,
    "Bob": 3.14,
    "Charlie": "Eight"
}

print(fav_colors["Alice"])
```

Parsing

- In either case
- We need a way of checking
- If we are looking at an expression
- Or just a literal
- We can be pretty loose with this
- Check if it contains a '+'
- (but you would likely want something more complex)

Handle Literals

- Right now, sees everything as str
- Try an int conversions [int()]
- Changes its *run-time* type
- If that fails: Strip quote, store as str

Handle Expressions

- Before we tackle this
- Let's make a dynamic add function
- Should handle "regular" adds
- Otherwise -> Concat to a string

Handle Expressions

- Let's use it
- Use spaces as a delimiter
- (Ignore + for now)
- Pass each to dynamic_add()

Done?

- With that, it should just work
- Do this for every line
- Grunt work -> Python handles how + should work
- Differs from a statically typed approach

Multiply *

- Time permitting, we can add *
- That operation IS well defined
- At least in python :)
- "str" * 3 -> "strstrstr"

Summary

- Feel the compromises of dynamic typing
- Hopefully you can envision:
- How different this would be in a static language
- More python practice

Questions